A Contracts Safety Theorems

Theorem A.1. (Contracts Type Safety)

1. If $\cdot \vdash e : \tau$ and $e \twoheadrightarrow e'$ then $\cdot \vdash e' : \tau$.
2. If $\cdot \vdash e : \tau$ then e is a value or $e = \mathsf{blame}_j^k$ or $e \twoheadrightarrow e'$ for some e'.

Proof. By Lemma A.2 and Lemma A.3. □

Lemma A.2. (Contracts Preservation) If $\Gamma \vdash e_1 : \tau$ and $e_1 \twoheadrightarrow e_2$ then $\Gamma \vdash e_2 : \tau$.

Proof of Lemma A.2. By cases on the step.

- Case Step-C, $E[e_1'] \twoheadrightarrow E[e_2']$ with $e_1' \dashrightarrow e_2'$.

  $\Gamma \vdash e_1' : \tau_1$ — Lemma A.7
  $\Gamma \vdash e_2' : \tau_1$ — Lemma A.8
  $\Gamma \vdash E[e_2'] : \tau$ — Lemma A.7

- Case Step-C-Blame, $E\left[\mathsf{blame}_j^k\right] \twoheadrightarrow \mathsf{blame}_j^k$.

  $\Gamma \vdash \mathsf{blame}_j^k : \tau$ — `Type-C-Blame`

□

Lemma A.3. (Contracts Progress)

If $\Gamma \vdash e_1 : \tau$ then e_1 is a value, or $e_1 = \mathsf{blame}_j^k$, or there is e_2 with $e_1 \twoheadrightarrow e_2$.

Proof of Lemma A.3. By induction on the derivation of $\Gamma \vdash e_1 : \tau$.

During induction, we encounter case $e' = \mathsf{blame}_{j'}^{k'}$. This makes $e = E\left[\mathsf{blame}_{j'}^{k'}\right]$ and would take a step. In the following proof, we left this case (blame) out.

- Cases of Base.

  Similarly to [?], except for T-App. Function type by Lemma A.4, introduces two new cases:

  - $e_3 = \mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)$. Steps by Guard-Func.
  - $e_3 = \mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda x.\kappa_2, v_1)$. Steps by Guard-Dep.

- Case Type-C-Blame. $e_1 = \mathsf{blame}_j^k$.

- Case Type-C-Guard. e_1 is a value.

- Case Type-C-Check, $e_1 = \mathsf{check}_j^k(e, v)$.

  $\Gamma \vdash e : \mathsf{bool}$ — Type-C-Check
  $e = \mathsf{true} \mid \mathsf{false}$ — Lemma A.4
  $e_1 \twoheadrightarrow e_1'$ — Check-True or Check-False

- Case Type-C-Mon, $e_1 = \mathsf{mon}_j^{k,l}(\kappa, v)$

  $\Gamma \vdash \kappa : \mathsf{con}(\tau)$ Type-C-Mon

  Subcase on type of conract:

  - Type-C-Flat. Steps by Mon-Flat.
  - Type-C-Fun or Type-C-DepFun. Steps by Mon-Guard.

□

Lemma A.4. (Canonical forms) If $\Gamma \vdash v : \tau$ then either:

- If $\tau = \mathsf{bool}$ then $v = \mathsf{true}$ or $v = \mathsf{false}$
- If $\tau = \tau_1 \to \tau_2$ then $v = \lambda x : \tau_1.e$ or $v = \mathsf{guard}_j^{k,l}(\kappa, v')$ where $\kappa = \kappa_1 \to \kappa_2$ or $\kappa = \kappa_1 \to_i \lambda y.\kappa_2$

Proof. Trivial by type inversion. □

Lemma A.5. (Substitution Expression)

If $\Gamma \vdash e_1 : \tau_1$ and $\Gamma, x : \tau_1 \vdash e : \tau$ then $\Gamma \vdash e[x := e_1] : \tau$.

Proof. By induction on $\Gamma, x : \tau_1 \vdash e : \tau$. The cases of Base are those of [?].

- Case $\Gamma, x : \tau_1 \vdash \mathsf{mon}_j^{k,l}(\kappa, e_2) : \tau$

  $\Gamma, x : \tau_1 \vdash \kappa : \mathsf{con}(\tau)$ `Type-C-Mon`
  $\Gamma, x : \tau_1 \vdash e_2 : \tau$ `Type-C-Mon`
  $\Gamma \vdash \kappa[x := e_1] : \mathsf{con}(\tau)$ Lemma A.6
  $\Gamma \vdash e[x := e_1] : \tau$ I.H.
  $\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa, e)[x := e_1] : \tau$ `Type-C-Mon`

- Cases Type-C-Check and Type-C-Guard.

  Similarly to the above, using I.H..

- Case $\Gamma, x : \tau_1 \vdash \mathsf{blame}_j^k : \tau$

  $\mathsf{blame}_j^k[x := e_1] = \mathsf{blame}_j^k$ Substitution
  $\Gamma \vdash \mathsf{blame}_j^k[x := e_1] : \tau$

□

Lemma A.6. (Substitution Contract)

If $\Gamma \vdash \kappa : \tau_1$ and $\Gamma, x : \tau_1 \vdash \kappa : \mathsf{con}(\tau)$ then $\Gamma \vdash \kappa[x := e_1] : \mathsf{con}(\tau)$.

Proof.

- Case $\Gamma, x : \tau_1 \vdash \mathsf{flat}(e) : \mathsf{con}(\tau)$

  $\Gamma, x : \tau_1 \vdash e : \tau \to \mathsf{bool}$ `Type-C-Flat`

$\Gamma \vdash e[x := e_1] : \tau \to \mathsf{bool}$ — Lemma A.5
$\Gamma \vdash \mathsf{flat}(e)[x := e_1] : \mathsf{con}(\tau)$ — Type-C-Flat

- Case Type-C-Fun and Type-C-Tuple.

  Similarly to the above.

- Case $\Gamma, x : \tau_1 \vdash \kappa_1 \to_i \lambda y.\kappa_2 : \mathsf{con}(\tau_2 \to \tau_3)$, $y \neq x$ and $y \notin \mathrm{fv}(e_1)$.

  $\Gamma, x : \tau_1 \vdash \kappa_1 : \mathsf{con}(\tau_2)$ — Type-C-DepFun
  $\Gamma, x : \tau_1, y : \tau_2 \vdash \kappa_2 : \mathsf{con}(\tau_3)$ — Type-C-DepFun
  $\Gamma \vdash \kappa_1[x := e_1] : \mathsf{con}(\tau_2)$ — I.H.
  $\Gamma, y : \tau_2 \vdash e_1 : \tau_1$ — Weakening context
  $\Gamma, y : \tau_2 \vdash \kappa_2[x := e_1] : \mathsf{con}(\tau_3)$ — I.H.
  $\Gamma \vdash \kappa[x := e_1] : \mathsf{con}(\tau_2 \to \tau_3)$ — Type-C-DepFun

□

Lemma A.7. (Decomposition) If $\Gamma \vdash E[e] : \tau$ then there is τ_1 such that $\Gamma \vdash e : \tau_1$, and for every e' with $\Gamma \vdash e' : \tau_1$ we have $\Gamma \vdash E[e'] : \tau$.

Proof. By induction on E. The frames of Base are as in [?]. Two frames are new.

- Case $E = \mathsf{mon}^{k,l}_j(\kappa, E_0)$.

  $\Gamma \vdash \mathsf{mon}^{k,l}_j(\kappa, E_0[e]) : \tau$ — Given
  $\Gamma \vdash \kappa : \mathsf{con}(\tau)$ — Type-C-Mon
  $\Gamma \vdash E_0[e] : \tau$ — Type-C-Mon
  $\Gamma \vdash e : \tau_1$ — I.H.
  $\Gamma \vdash E_0[e'] : \tau$ — I.H., for $\Gamma \vdash e' : \tau_1$
  $\Gamma \vdash \mathsf{mon}^{k,l}_j(\kappa, E_0[e']) : \tau$ — Type-C-Mon

- Case $E = \mathsf{check}^k_j(E_0, v)$.

  $\Gamma \vdash \mathsf{check}^k_j(E_0[e], v) : \tau$ — Given
  $\Gamma \vdash E_0[e] : \mathsf{bool}$ — Type-C-Check
  $\Gamma \vdash v : \tau$ — Type-C-Check
  $\Gamma \vdash e : \tau_1$ — I.H.
  $\Gamma \vdash E_0[e'] : \mathsf{bool}$ — I.H., for $\Gamma \vdash e' : \tau_1$
  $\Gamma \vdash \mathsf{check}^k_j(E_0[e'], v) : \tau$ — Type-C-Check

□

Lemma A.8. (Redex preservation) If $\Gamma \vdash e_1 : \tau$ and $e_1 \dashrightarrow e_2$ then $\Gamma \vdash e_2 : \tau$.

Proof. By cases on $e_1 \dashrightarrow e_2$. The reductions of Base are as in [?], using Lemma A.5 for S-App and S-Fix.

- Case $\mathsf{check}^k_j(\mathsf{true}, v) \dashrightarrow v$.

  $\Gamma \vdash v : \tau$ Type-C-Check

- Case $\mathsf{check}^k_j(\mathsf{false}, v) \dashrightarrow \mathsf{blame}^k_j$.

  $\Gamma \vdash \mathsf{blame}^k_j : \tau$ Type-C-Blame

- Case $\mathsf{mon}^{k,l}_j(\mathsf{flat}(e), v) \dashrightarrow \mathsf{check}^k_j(e\ v, v)$.

  $\Gamma \vdash \mathsf{flat}(e) : \mathsf{con}(\tau)$ Type-C-Mon
  $\Gamma \vdash v : \tau$ Type-C-Mon
  $\Gamma \vdash e : \tau \to \mathsf{bool}$ Type-C-Flat
  $\Gamma \vdash e\ v : \mathsf{bool}$ T-App
  $\Gamma \vdash \mathsf{check}^k_j(e\ v, v) : \tau$ Type-C-Check

- Case $\mathsf{mon}^{k,l}_j(\kappa, v) \dashrightarrow \mathsf{guard}^{k,l}_j(\kappa, v)$ with κ a function contract.

  $\Gamma \vdash \kappa : \mathsf{con}(\tau)$ Type-C-Mon
  $\Gamma \vdash v : \tau$ Type-C-Mon
  $\Gamma \vdash \mathsf{guard}^{k,l}_j(\kappa, v) : \tau$ Type-C-Guard

- Case $\mathsf{guard}^{k,l}_j(\kappa_1 \to \kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}^{k,l}_j\left(\kappa_2, v_1\ \mathsf{mon}^{l,k}_j(\kappa_1, v_2)\right)$.

  $\Gamma \vdash v_2 : \tau_1$ T-App
  $\Gamma \vdash \mathsf{guard}^{k,l}_j(\kappa_1 \to \kappa_2, v_1) : \tau_1 \to \tau$ T-App
  $\Gamma \vdash \kappa_1 \to \kappa_2 : \mathsf{con}(\tau_1 \to \tau)$ Type-C-Guard
  $\Gamma \vdash v_1 : \tau_1 \to \tau$ Type-C-Guard
  $\Gamma \vdash \kappa_1 : \mathsf{con}(\tau_1)$ Type-C-Fun
  $\Gamma \vdash \kappa_2 : \mathsf{con}(\tau)$ Type-C-Fun
  $\Gamma \vdash \mathsf{mon}^{l,k}_j(\kappa_1, v_2) : \tau_1$ Type-C-Mon
  $\Gamma \vdash v_1\ \mathsf{mon}^{l,k}_j(\kappa_1, v_2) : \tau$ T-App
  $\Gamma \vdash \mathsf{mon}^{k,l}_j\left(\kappa_2, v_1\ \mathsf{mon}^{l,k}_j(\kappa_1, v_2)\right) : \tau$ Type-C-Mon

- Case Guard-Dep, $\mathsf{guard}^{k,l}_j(\kappa_1 \to_i \lambda x.\kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}^{k,l}_j\left(\kappa_3, v_1\ \mathsf{mon}^{l,k}_j(\kappa_1, v_2)\right)$ with $\kappa_3 = \kappa_2\left[x := \mathsf{mon}^{l,j}_j(\kappa_1, v_2)\right]$.

  $\Gamma \vdash v_2 : \tau_1$ T-App
  $\Gamma \vdash v_1 : \tau_1 \to \tau$ Type-C-Guard
  $\Gamma \vdash \kappa_1 \to_i \lambda x.\kappa_2 : \mathsf{con}(\tau_1 \to \tau)$ Type-C-Guard
  $\Gamma \vdash \kappa_1 : \mathsf{con}(\tau_1)$ Type-C-DepFun
  $\Gamma, x : \tau_1 \vdash \kappa_2 : \mathsf{con}(\tau)$ Type-C-DepFun

$\Gamma \vdash \mathsf{mon}_j^{l,j}(\kappa_1, v_2) : \tau_1$ Type-C-Mon

$\Gamma \vdash \kappa_3 : \mathsf{con}(\tau)$ Lemma A.6

$\Gamma \vdash \mathsf{mon}_j^{l,k}(\kappa_1, v_2) : \tau_1$ Type-C-Mon

$\Gamma \vdash v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2) : \tau$ T-App

$\Gamma \vdash \mathsf{mon}_j^{k,l}\left(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right) : \tau$ Type-C-Mon

□